

AI 协作反思日志 — Phase 3 详细设计

团队与阶段

项目	内容
团队	Git Guild 项目组
阶段	Phase 3 详细设计
日期	2026-05-15
本阶段目标	完成类图、SOLID 检查、API 规范、数据库设计和整合版详细设计

1. 本阶段 AI 使用清单

任务	AI 工具	Prompt 摘要	AI 输出质量 (1-5)	人工修改幅度
核心类图初稿	Codex	根据 P1 需求和 P2 架构生成核心类图，并覆盖 P0 模块	3	60%
SOLID 检查	Codex	从 S/O/L/I/D 五个维度审查 AI 类图设计	4	35%
API 规范草案	Codex	按 RESTful 风格生成注册登录、任务、审核、推荐等接口规范	3	50%
数据库设计	Codex	根据类图 and 业务流程生成 ER 图、建表 SQL、索引和隐私说明	4	40%
文档整合	Codex	将类图、API、数据库和 AI 审查记录整理为 P3 详细设计文档	4	30%

2. AI 助力分析 (Q1)

AI 在本阶段最有帮助的地方是快速铺开结构。类图、API 清单、错误码、SQL 表结构这些内容如果完全手写，容易遗漏模块；让 AI 先生成一版后，团队可以把精力放在审查边界、修正命名和确认业务规则上。

具体帮助包括：

快速列出 P0 模块对应的领域类，帮助团队发现任务分类、任务标签、管理员审核、推荐结果等不应遗漏的对象。

快速生成 API 文档骨架，让每个接口都有 URL、方法、参数、响应和错误码。

快速生成 SQL 初稿，便于团队围绕范式、索引和隐私字段做审查。

帮助统一文档格式，让 P3 交付物更容易检查和维护。

3. AI 误导分析（Q2）

SOLID 检查中发现的 AI 设计问题数量：15 处。

最严重的设计问题是：AI 容易把实体对象、业务服务和外部平台调用混在一起。例如原始设计倾向于让 `Repository` 自己负责导入和同步，让 `Notification` 自己负责发送，让 `User` 承担登录和外部账号绑定行为。这会让实体类职责过重，也会让核心业务直接依赖 GitHub 或 Gitea 的具体 API。

本阶段发现的主要误导包括：

AI 容易默认使用 GitHub OAuth，但项目已经决定采用平台自有账号体系。

AI 容易把推荐系统写成附加功能，但项目中推荐匹配是核心能力。

AI 生成 API 时常忽略失败响应、权限校验和参数边界。

AI 生成数据库时容易缺少索引说明，也容易把分类、标签等字段直接冗余到任务表中。

AI 对“订单 / 需求 / 任务”的领域语言不敏感，需要人工统一为项目内的“任务 / 悬赏任务”。

4. Prompt 改进计划（Q3）

上阶段改进措施的验证结果：要求 AI 明确引用 P1/P2 文档后，输出内容与项目实际目标更接近，尤其是平台自有账号、Gitea 外部支撑和推荐核心化这几个决策没有再次严重偏离。

本阶段新的改进措施：

在 Prompt 中先声明项目领域语言，例如统一使用“任务 / 悬赏任务”，不混用“订单”。

要求 AI 输出前先列出假设，再生成设计，避免它默认引入 GitHub OAuth、微服务或复杂中间件。

对 API 和数据库设计，要求 AI 必须包含失败路径、权限规则、索引和隐私字段处理。

对类图设计，要求 AI 分离实体、服务、接口和外部适配器，避免把行为全部塞进实体类。

5. 本阶段的核心工程判断

决策内容：P3 详细设计继续沿用 P2 的模块化单体方案，并把外部代码托管能力收敛在代码支撑平台适配模块中；推荐匹配、管理员任务审核、任务分类与筛选都作为核心业务设计，不后置为可选扩展。

为什么 AI 无法单独做这个决策：这个判断依赖课程周期、团队实现能力、P1 访谈结果、P2 架构约束和小组讨论后的业务取舍。AI 可以给出通用技术建议，但无法真正知道团队的时间、人力和验收优先级，因此最终需要人工决定哪些功能必须进入 P0，哪些功能只能作为后续增强。

6. 人工修正记录

修正点	AI 初稿倾向	人工处理
账号体系	倾向 GitHub OAuth	改为平台自有账号，GitHub 只作为仓库导入来源
类职责	实体类承担同步、认证、发送等行为	拆为实体、服务、适配器、策略接口
推荐系统定位	容易被写成扩展能力	明确推荐匹配是 P0 核心能力
管理员审核	容易被遗漏或写成后台附属功能	单独设计管理员审核记录和管理员审核 API
API 错误处理	成功响应较完整，失败响应不足	为每组 API 补充错误码、HTTP 状态码和典型失败响应
数据库索引	只给出主键和少量外键	补充任务筛选、推荐结果、通知中心、成长记录等高频查询索引
隐私数据	容易出现 password、外部 token 等敏感字段	改为 password_hash，外部 token 不进入业务表

7. 阶段结论

AI 在详细设计阶段适合作为“结构生成器”和“审查辅助工具”，不适合直接作为最终架构决策者。最终设计质量主要来自人工对需求、架构边界、课程约束和实现成本的判断。后续实现阶段可以继续让 AI 辅助生成代码和测试，但需要保持同样的边界审查：先确认领域语言和模块职责，再让 AI 写具体实现。